

UE23CS352A: Machine Learning Hackathon

Hackman

Section F | Team no.12 | 03-11-2025

Meenakshi Pathiyil	PES2UG23CS335
Neema Shrivastava	PES2UG23CS377
Nidhi K	PES2UG23CS383
Navya G N	PES2UG23CS372

Output

main_evaluate.py

```
(venv) meenakshi@Meenakshis-MacBook-Pro Hackman % python3 main_evaluate.py
Evaluating on 4003 test words
Loaded DQN model

EVALUATION RESULTS (2000 games)
Success Rate: 11.5%
Total Wrong Guesses: 11524
Total Repeated Guesses: 0
Final Score: -57390.0
```

Key Observations: What were the most challenging parts? What insights did you gain?

The most challenging aspect of this hybrid system was the integration and effective coupling of the HMM and the RL agent. The HMM training itself presented a challenge in determining the optimal number of hidden states (N=40 was chosen after experimentation) and handling words of varying lengths by using start/end tokens and concatenation. The primary insight gained was that a simple letter frequency approach is inadequate; the HMM's probabilistic context is essential for competitive performance, providing a probability distribution that acts as a strong, informed prior for the RL decision-maker. Furthermore, Reward Shaping was more critical than anticipated, as penalties for repeated guesses (-5) and wrong guesses (-10) had to be carefully balanced against the win bonus (+100) and correct guess reward (+10) to guide the agent toward the goal of maximum efficiency (fewest mistakes), not just winning.

Strategies: Discuss your HMM design choices. Detail your RL state and reward design and why you chose them.

HMM Design

We chose a Categorical Hidden Markov Model with 30 hidden states and 28 emissions (26 letters + <S> and <E> tokens). This design was a strategic compromise: it is complex enough to capture basic first-order dependencies (like which letter follows a vowel or precedes an 'E'), yet simple enough to be trained robustly on the 50,000-word corpus without excessive computational cost. The use of start and end tokens was necessary to handle the context of the beginning and end of a word, effectively normalizing sequences of different lengths for a single HMM.

RL State and Reward Design

The RL agent uses a Deep Q-Network (DQN) because the resulting state vector size is 377, far too large for a traditional Q-table approach.

- **State:** The state vector is a hybrid that combines raw game data with the HMM's linguistic intelligence:
 - Masked Word (324 dims): Provides the specific pattern/length context.
 - Guessed Letters (26 dims): Crucial for preventing repeated, illegal actions.
 - Lives Left (1 dim): Provides an urgency/risk signal to the agent.
 - HMM Probabilities (26 dims): The most critical piece, directly informing the agent which letters are linguistically most likely to complete the current pattern.
- **Reward:** The reward function was designed to mirror the final scoring metric, maximizing success while minimizing inefficiency:
 - $R(\text{Win}) = +100$
 - $R(\text{Correct Guess}) = +10$
 - $R(\text{Wrong Guess}) = -10$
 - $R(\text{Repeated Guess}) = -5$

Exploration: How did you manage the exploration vs. exploitation trade-off?

We managed the trade-off using a standard ϵ -greedy strategy with an epsilon decay schedule.

- Exploration: The agent chose a random, un-guessed letter with probability ϵ , allowing it to discover unexpected or non-intuitive letter combinations that the HMM might overlook.
- Decay Schedule: ϵ was initialized high at 0.5 to encourage broad exploration across the state space during the early phases of the 1000-episode training.
- Exploitation: ϵ was progressively decayed down to a minimum of 0.1. This shift allowed the agent to rely more heavily on its learned Q-values (exploitation) in later episodes, solidifying a robust final policy. This balancing act was key to ensuring the agent learned the optimal policy without getting stuck in local optima.

Future Improvements: If you had another week, what would you do to improve your agent?

Contextual HMM Integration: The current HMM provides marginal probabilities for only the first blank spot. A superior approach would be to calculate the marginal probability distribution for *every* blank spot and include this full matrix (e.g. 12 x 26 dimensions) in the state vector. This would allow the RL agent to reason about the relative likelihood of a letter appearing in position 3 vs. position 7, substantially improving its predictive power.

Target Network for DQN: Implementing a separate Target Network (a common DQN technique) would stabilize the training process. The target Q-values used in the Bellman equation would be calculated using this static target network, reducing oscillation and improving convergence compared to the current single-network approach.

Prioritized Experience Replay (PER): Instead of sampling uniformly from the replay memory, we would implement PER to prioritize the sampling of "surprising" or high-error transitions (those with a high Temporal Difference error). This focuses the agent's learning efforts on the most informative experiences, which is particularly beneficial in games with sparse, delayed rewards like Hangman.